
LLM Eval CI/CD Workstation

Complete Technical Documentation & Engineering Session Log

What the project is, every tool it uses and why, how the whole system works end to end, and a full step-by-step record of everything built, fixed, and diagnosed in this session.

Project: **llm-eval-pipeline** | Owner: Anshi Tyagi | Repository: /solar
Prepared by Claude (Anthropic) working session log

Table of Contents

- 1. Executive Summary — What This Project Is
- 2. Why This Project Matters
- 3. System Architecture — How Everything Connects
- 4. Every Tool & Technology Used, and What It Does
- 5. The Golden Dataset & Quality Gate Metrics
- 6. The Four Application Views, In Detail
- 7. Full Step-by-Step Session Log
 - 7.1 Initial Codebase Audit
 - 7.2 Round 1: Three Targeted Fixes
 - 7.3 The Full “Microsoft-Level” Upgrade (TypeScript Migration & Beyond)
 - 7.4 Vercel Deployment Diagnosis & Fix
- 8. Known Limitations & Honest Caveats
- 9. How to Run Everything (Command Reference)
- 10. Final Assessment

1. Executive Summary — What This Project Is

The **LLM Eval CI/CD Workstation** is a self-contained LLMOps evaluation platform. It answers a question that traditional software testing already solved decades ago, but that most teams building LLM-powered applications still handle by hand: *“If I change this prompt, swap this model, or tweak my RAG retrieval settings, did I just make the system worse?”*

It does this by treating prompt/RAG configuration changes exactly like code changes: every change is run against a fixed 100-item “golden” benchmark dataset, scored on five metrics (hallucination rate, faithfulness, p95 latency, answer relevancy, and cost), and checked against hard Service Level Agreement (SLA) thresholds. If any threshold is breached, the pipeline exits with a non-zero status code — the same mechanism a failing unit test uses to block a merge in any conventional CI system.

The project ships three things that work together: (1) a **CLI gatekeeper script** designed to run in git hooks or GitHub Actions, (2) an **interactive React dashboard** for humans to explore results, tweak parameters, and trigger runs by hand, and (3) a small **Express/SQLite backend** that persists run history and proxies real calls to Google's Gemini API so that a client-side API key is never baked into the built frontend bundle.

A defining, unusually honest feature of this project is its **Transparency Log**: it is explicit, in both the UI and the documentation, about which code paths are **simulated** (mathematical heuristics standing in for a real model, used for four “mock” model options) and which are **real** (actual network calls to Gemini 1.5 Flash, including a genuine two-step LLM-as-a-judge grading chain). Nothing is dressed up to look more “real” than it is.

2. Why This Project Matters

Traditional software is deterministic: the same input to the same code always produces the same output, which is exactly what makes unit tests meaningful. Large Language Models break this assumption. The exact same prompt sent to the exact same model can produce a different answer on every call, and even small, well-intentioned edits — rewording a system instruction, changing a RAG chunk size from 500 to 1000 characters, switching from GPT-4o-mini to GPT-3.5-turbo to save cost — can silently introduce hallucinations, blow past a latency budget, or degrade answer quality in ways that are invisible until a user notices.

This project's core value proposition is turning that invisible risk into an automated, visible gate:

- **Continuous Quality Gates** — a prompt change that pushes the hallucination rate above 5% is treated exactly like a broken unit test: the build fails and the merge is blocked.
- **Systematic Benchmarking** — every run is scored against the same 100-item dataset spanning Customer Support, Technical & Coding, Financial & Data Extraction, and Legal & Document Analysis categories, so results are comparable across runs instead of anecdotal.
- **Regression Auditing** — the Dual-Run Comparator lets a developer pick any two historical commits and see, side by side, exactly which metric moved and by how much.
- **Interactive Experimentation** — the CI Run Simulator lets a developer iterate on a prompt template and RAG parameters live, watching a scrolling terminal-style console report results in real time, before ever touching a real commit.

3. System Architecture — How Everything Connects

The system has four moving parts that all read and write the same evaluation data, from two different entry points (a human using the dashboard, or an automated CI trigger):

Component	Location	Role
Golden Dataset	src/data/golden_dataset.json	A static, version-controlled JSON file of 100 question/expected-answer/reference-context triples across 4 categories. This is the fixed benchmark every run is scored against — it never changes as part of a normal evaluation run.
CLI Runner	scripts/eval-runner.ts	A standalone Node/TypeScript script (no browser involved). Parses CLI flags, loads the golden dataset, evaluates each item (either simulated or via a real Gemini call), aggregates metrics, checks them against the SLA gates, writes the result into SQLite, and exits 0 (pass) or 1 (fail).
Web Dashboard	src/ (React + Vite + TypeScript)	A 4-tab single-page app: Metrics Control Room (Dashboard), CI Run Simulator, Dual-Run Comparator, and Golden Dataset explorer. Talks to the backend over four REST endpoints. Runs the identical scoring logic as the CLI (via a module both share) when operating in simulated mode.
Backend API	server.ts + database.ts (Express + SQLite)	A small Express app exposing /api/config, /api/runs, /api/runs/add, and /api/eval. It is the only thing that ever holds a Gemini API key server-side, and the only thing that talks to the SQLite file directly.

End-to-end flow (automated CI path):

- 1 A developer changes a prompt template, model choice, or RAG parameter, and commits.
- 2 A CI trigger (GitHub Actions, or a local git hook) runs `npm run eval`, invoking `scripts/eval-runner.ts`.
- 3 The runner loads `golden_dataset.json` and iterates over the requested subset of test cases.
- 4 For each case, it either (a) computes a simulated score using hyperparameter-correlated heuristics, or (b) makes two real calls to Gemini 1.5 Flash — one to generate an answer, one to have Gemini itself grade that answer's faithfulness and relevancy against the retrieved context.
- 5 Per-case results are aggregated into run-level metrics: hallucination rate, average relevancy, average faithfulness, p50/p95 latency, and total cost.
- 6 Those metrics are checked against the three hard SLA gates. If any gate fails, the process exits with code 1, which a CI system (or a git hook) treats as a failed check and blocks the merge.
- 7 Either way, the full result — pass or fail — is written to the SQLite `runs` and `test_results` tables via `database.ts`.
- 8 The next time anyone opens the web dashboard, `GET /api/runs` returns the full updated history, and the new commit shows up in the trend chart and the comparator.

End-to-end flow (interactive path, via the Simulator tab):

Identical in spirit, but driven from the browser: `src/hooks/useEvalRuns.ts` runs the same scoring logic in the browser (importing the exact same `src/utils/evalEngine.ts` module the CLI uses, so the two can never drift apart), and for real Gemini calls it goes through the backend's `/api/eval` proxy rather than calling Google directly — this is what keeps a pasted API key from ever being visible in a network request that Google itself could see logged from the client, and prevents it from ever being compiled into the static frontend bundle.

4. Every Tool & Technology Used, and What It Does

Every dependency in this project earns its place — nothing here is decorative. The table below covers every tool touched during this project's life, grouped by concern.

Frontend

Tool	What it does here
React 18	Component-based UI library. Powers all four tabs (Dashboard, Simulator, Comparator, Dataset) as function components using hooks (<code>useState</code> , <code>useEffect</code> , <code>useRef</code>).
Vite	Frontend build tool and dev server. Provides instant hot-module-reload during development on port 5188, proxies any <code>/api/*</code> request to the backend on port 5189, and produces the optimized production bundle in <code>dist/</code> via <code>vite build</code> .
TypeScript	Statically-typed superset of JavaScript. Every file in the project — frontend, backend, CLI scripts, and tests — is written in TypeScript with <code>strict</code> mode on, so the compiler catches type mismatches (wrong prop shapes, missing fields, wrong function signatures) before the code ever runs.
Recharts	A React charting library built on D3. Renders the “SLA Performance & Quality Trends Across Commits” line chart on the Dashboard tab, plotting faithfulness and hallucination rate across every historical run, with separate solid/dashed lines for real vs. simulated data.
Lucide React	An icon component library. Supplies every icon in the UI — navigation icons, the search/key/play/database glyphs, status check/cross marks, and more.

Backend

Tool	What it does here
Express 5	Minimal Node.js web framework. Defines the four REST routes (<code>/api/config</code> , <code>/api/runs</code> , <code>/api/runs/add</code> , <code>/api/eval</code>), the rate-limiting middleware, and the two authentication middlewares. Also serves the built static frontend in production.
sqlite3 (npm package)	Node bindings for SQLite, an embedded, file-based relational database. Stores two tables: <code>runs</code> (one row per evaluation run) and <code>test_results</code> (one row per individual test case within a run, foreign-keyed to its parent run with cascading delete).
Node.js native TypeScript execution	This project's backend and CLI scripts run directly as <code>.ts</code> files via Node's built-in type-stripping support (<code>node server.ts</code> , <code>node scripts/eval-runner.ts</code>) — no separate compile step or <code>ts-node</code> / <code>tsx</code> dependency is needed to run them.

Testing

Tool	What it does here
Vitest	A Vite-native test runner, API-compatible with Jest. Runs all unit and integration tests via <code>npm run test</code> : scoring/gating logic, SQLite behavior, and full HTTP route behavior.

Tool	What it does here
Supertest	An HTTP assertion library that can drive an Express app in-process, without binding it to a real network port. Used in <code>tests/server.test.ts</code> to send real HTTP requests (GET/POST, with/without auth headers, with malformed bodies) straight at the exported Express app and assert on the response.

Code Quality & Git Automation

Tool	What it does here
ESLint (flat config) + typescript-eslint	Static analysis linter. Configured in <code>eslint.config.js</code> to parse TypeScript and JSX, catch unused variables/imports, and enforce React's Rules of Hooks.
eslint-plugin-react-hooks / eslint-plugin-react-refresh	ESLint plugins specifically for React: the former enforces the Rules of Hooks (hooks only called at the top level, in the same order every render); the latter warns about patterns that break Vite's Fast Refresh (hot reload).
Husky	Manages Git hooks from within the npm project (installed automatically via the prepare script). Runs <code>typecheck + lint</code> before every commit, and <code>test + a real eval gate</code> run before every push.

CI/CD, Deployment & Documentation

Tool	What it does here
GitHub Actions	Cloud CI/CD platform. The workflow at <code>.github/workflows/eval-gate.yml</code> runs on every push/PR to main: install, typecheck, lint, test, build, then the real eval SLA gate — and posts the gate's pass/fail result as a comment on the pull request.
Docker & Docker Compose	Containerization. A multi-stage <code>Dockerfile</code> builds the frontend in one stage and runs the Express server in a slim runtime stage; <code>docker-compose.yml</code> wires it up behind a single <code>docker compose up --build</code> command, with a named volume so the SQLite file survives container restarts.
OpenAPI 3.0	An industry-standard machine-readable API specification format. <code>openapi.yaml</code> documents all four backend routes — parameters, request/response schemas, status codes, and the auth model — and is validated clean by <code>@redocly/cli lint</code> .
Vercel	The serverless hosting platform the live demo (<code>solar-six-tan.vercel.app</code>) runs on. Requires explicit configuration (added this session — see Section 7.4) to route API traffic to the Express app instead of 404ing at the platform edge.
PDFKit	A Node.js PDF-generation library. <code>scripts/generate-pdf.ts</code> uses it to programmatically draw the original <code>PROJECT_EXPLANATION.pdf</code> — titles, flowchart boxes/arrows/diamonds, and paragraphs — entirely in code (no design tool involved).

Tool	What it does here
This document, PROJECT_EXPLANATION_DETILED.pdf, was generated the same way this session, using Python's ReportLab library instead, since the request was made outside the Node/TypeScript toolchain.	

5. The Golden Dataset & Quality Gate Metrics

`src/data/golden_dataset.json` contains 100 fixed test cases split evenly across four categories — Customer Support, Technical & Coding, Financial & Data Extraction, and Legal & Document Analysis. Each item has an `id`, a `question`, an `expected_answer`, and a `reference_context` (the grounding document a RAG system would have retrieved). Every evaluation run scores some subset of these same 100 items, which is what makes runs comparable to each other over time.

Metric	SLA Threshold	What it measures
Hallucination Rate	$\leq 5\%$	Fraction of test cases where the model's answer asserts something not supported by the reference context.
Faithfulness	$> 90\%$	How strictly the answer is grounded in the retrieved context, versus drawing on the model's own pretrained knowledge.
p95 Response Latency	$\leq 2000\text{ms}$	95th-percentile response time across the scored subset — protects against tail-latency regressions a simple average would hide.
Answer Relevancy	Informational	Keyword/semantic overlap between the generated answer and the expected ground truth. Tracked and displayed, but does not block a merge on its own.
Eval Cost	Informational	Total estimated API cost for the run, based on input/output token counts and per-model pricing — flags sudden cost regressions without gating the build.

Two scoring paths, computed by the same shared module (`src/utils/evalEngine.ts`):

- **Simulated path** (model options: `gpt-4o`, `gpt-4o-mini`, `claude-3-5-sonnet`, `gpt-3.5-turbo`) — no network call happens at all. Instead, `simulateTestResult()` computes believable latency, cost, faithfulness, relevancy, and hallucination values using per-model base rates plus hyperparameter correlations (e.g. a prompt containing safety language like “rely only on” halves the simulated hallucination rate; a chunk size under 200 characters increases it). This exists purely to make the dashboard's charts and gates exercisable without spending real API credits.
- **Real path** (model option: `gemini-1.5-flash`) — makes two genuine HTTPS calls to Google's Gemini API through the backend proxy. Call 1 sends the prompt template (with `{{context}}` and `{{question}}` substituted) and gets back a real generated answer. Call 2 is the **LLM-as-a-judge** step: a second, independent prompt containing the question, the context, and the model's own answer is sent back to Gemini, instructing it to output a strict JSON object with `faithfulness`, `relevancy`, and `isHallucinating` fields — using Gemini's structured-output (`responseSchema`) feature so the judgment is guaranteed valid JSON rather than free text that would need fragile parsing.

If a real-API call fails for any reason (bad key, rate limit, network error, malformed response), the runner is designed to **fail loudly**: it prints the full error to the console/terminal log and aborts immediately with a non-zero exit code, rather than silently masking the failure and reporting fake success.

6. The Four Application Views, In Detail

6.1 Metrics Control Room (Dashboard)

The landing tab. Shows a “commit ribbon” banner for the most recent run (commit id, model, author, pass/fail badge, simulated-vs-real indicator), four animated circular SVG gauges for hallucination rate, faithfulness, p95 latency, and total cost, a Recharts line-trend chart plotting faithfulness and hallucination rate across every historical commit (with separate solid lines for real-API runs and dashed lines for simulated ones), and a searchable/filterable table of every individual test case in the active run. Clicking any row slides open a right-hand inspector drawer showing the full question, reference context, model output, and per-case latency/faithfulness.

6.2 CI Run Simulator

A two-column workstation: a hyperparameter form on the left (target model, prompt template textarea with `{{context}}`/`{{question}}` placeholders, RAG chunk size, top-K, evaluation subset size, and a commit message field) and a retro scrolling terminal console on the right that streams live, color-coded log lines as the run progresses. Selecting the “Gemini 1.5 Flash (REAL API)” option reveals a password-style input for a Bring-Your-Own-Key Google AI Studio key (or shows a green “server key active” badge if the server already has one configured), and clicking “Commit Change & Run Pipeline” actually executes the evaluation and persists the result to the database exactly like the CLI runner does.

6.3 Dual-Run Comparator

Pick any two historical commits from two dropdowns and see them rendered side by side: prompt templates, pass/fail badges, simulated-vs-real source badges, and every metric — with the right-hand column showing a colored delta badge (green if the change was an improvement, red if it was a regression) next to each metric, computed by the `calcDiff()` helper.

6.4 Golden Dataset Explorer

A read-only, searchable, category-filterable table of all 100 benchmark items — question, expected ground-truth answer, and reference context — so a developer can inspect exactly what the benchmark is testing before trusting a pass/fail result.

7. Full Step-by-Step Session Log

This section is the complete, chronological record of everything reviewed, diagnosed, fixed, and built during this engineering session, in the order it actually happened.

7.1 Initial Codebase Audit

The session began with a full read-through of the existing project (README, PROJECT_EXPLANATION.md, server.js, database.js, scripts/eval-runner.js, all React components, judgeParser.js, and the test suite) to understand what it does before touching anything. That audit surfaced six concrete findings:

- **npm run lint was completely broken** — the script invoked `eslint`, but ESLint wasn't even installed as a dependency and no config file existed.
- **The write endpoint had no real security by default** — `/api/runs/add`'s auth check only enforced a bearer token if `API_SECRET` happened to be set; if it wasn't configured on the live deployment, anyone on the internet could insert fake historical run records into the shared demo database.
- **The in-memory rate limiter was largely theater on serverless** — its state lives in a plain object that resets on every cold start on a platform like Vercel, and it never evicts old IPs (a slow memory leak on any long-lived instance).
- **Significant logic duplication** — the entire simulated-heuristic scoring formula, the real-Gemini two-step call sequence, the judge prompt text, and the SLA gating logic were copy-pasted nearly verbatim between `scripts/eval-runner.js` and `src/hooks/useEvalRuns.js` — any future formula or threshold change would have had to be made twice, and would silently drift if someone forgot.
- **Thin test coverage** — only one test file existed, covering the judge-output parser; the actual scoring math, gating logic, and every Express route and SQLite function had zero test coverage.
- **A UI copy/behavior mismatch** — the Simulator's own on-screen text claimed the Gemini key was “never stored... in client-side queries,” while the component's own code immediately called `localStorage.setItem('gemini_api_key', ...)`, persisting it in plaintext indefinitely.

7.2 Round 1: Three Targeted Fixes

After presenting the audit and getting agreement to act on it, three concrete fixes were implemented and verified:

1. Fail-closed authentication on the write endpoint

Added a second middleware, `requireAuthCheck`, used only on `POST /api/runs/add`. Unlike the original `authCheck` (kept as-is for `/api/eval`, which is intentionally open for anonymous Bring-Your-Own-Key demo use), the new middleware returns 503 and refuses the request outright if `API_SECRET` isn't configured, instead of silently letting the write through. Verified live with curl: no secret configured → 503; wrong/missing token with a secret configured → 403; correct token → passes through to the real logic.

2. A working ESLint setup

Installed `eslint`, `@eslint/js`, and `globals`, then wrote a real flat-config file (`eslint.config.js`), scoped deliberately to the long-standing, broadly-agreed `react-hooks/rules-of-hooks` and `react-hooks/exhaustive-deps` rules rather than the newer React-Compiler-oriented rule set that ships as “recommended” in the latest `eslint-plugin-react-hooks` major version (that stricter set would have

demanded an unrelated Suspense/data-fetching rewrite for a perfectly normal fetch-on-mount pattern). Roughly 30 real pre-existing warnings and errors were cleaned up along the way: dead imports, dead props, genuine no-useless-assignment bugs, and bare catch bindings.

3. Extracted the shared evalEngine module

Created `src/utils/evalEngine.js` (later converted to `.ts`) containing the simulation heuristics, the judge-prompt template builder, the per-test-case status function, metric aggregation, and SLA gate evaluation — all in one place. Both `scripts/eval-runner.js` and `src/hooks/useEvalRuns.js` were rewritten to import from it instead of duplicating the logic, and 10 new unit tests were added covering the gating math specifically (a category that had zero coverage before).

After these three fixes, a rating pass was given: 7.5/10 (Concept & scope 9, Code quality 7, Security 7, Testing 6, Documentation 8) — solid but with a real gap: `npm run lint` being broken and no actual CI wiring meant the project's central claim — being a “CI/CD gatekeeper” — wasn't literally true yet; it was an un-triggered local script.

7.3 The Full “Microsoft-Level” Upgrade

Asked what it would take to make this project genuinely interview-ready at a rigorous engineering bar, a tiered roadmap was proposed and, on approval, executed in full — an 11-part body of work, tracked explicitly as a task list from start to finish. Full TypeScript migration was confirmed as the deepest, most invasive item after an explicit choice was offered (full migration vs. lightweight JSDoc-only typing vs. skipping it) — full migration was chosen.

Step 1 — TypeScript tooling foundation

- Installed `typescript`, `@types/node`, `@types/react`, `@types/react-dom`, and `@types/express` (later also `@types/pdfkit` and `@types/supertest`).
- Set up a project-references style TypeScript configuration: `tsconfig.json` (a lightweight solution file), `tsconfig.app.json` (the browser/React side — DOM lib, JSX, includes `src/`), and `tsconfig.node.json` (the backend/CLI/test side — Node lib, includes `server.ts`, `database.ts`, `api/`, `scripts/`, and `tests/`).
- Confirmed empirically that this Node runtime executes `.ts` files natively (type-stripped, no separate compile step) — including resolving relative imports between two `.ts` files by their explicit `.ts` extension — so no `tsx/ts-node` runtime dependency was needed.
- Updated every relevant `package.json` script: added `typecheck`, made `build` run the `typecheck` before `vite` `build`, pointed `eval` and `start` at the `.ts` entry files, and added a `prepare` script for Husky.
- Converted `vite.config.js` to `.ts` (valid as-is, no content changes needed).

Step 2 — Shared domain types & utils

Created `src/types.ts` as the single shared domain model — `RagConfig`, `TestResult`, `RunMetrics`, `EvalRun`, `JudgeResult`, `GateResult`, `GoldenDatasetItem`, `ConsoleLogEntry`, and `SimulatorRunConfig` — imported by both the frontend and backend so the two sides of the API can never silently disagree about a field's shape. Converted `evalEngine.js` and `judgeParser.js` to fully typed `.ts`.

Step 3 — Backend: `database.ts`, `server.ts`, and a real validation fix

Converted `database.js` → `database.ts` with typed SQLite row interfaces and a typed `sqlite3.RunResult` callback (`this.lastID`). Converted `server.js` → `server.ts` with typed Express `Request/Response/NextFunction` handlers. Along the way, added a real fix that had been flagged but not yet built in Round 1: a `validateRunPayload()` function that checks `commitId`, `model`, the full shape of `ragConfig` (`chunkSize/chunkOverlap/topK`), the full shape of `metrics` (all seven numeric fields), and that `testResults` is actually an array — so a malformed POST body now returns a clean 400 with a specific field-level message instead of crashing to a raw 500. Verified live: a payload missing `ragConfig` now returns `{"error": "Missing or invalid ragConfig"}` with status 400.

Step 4 — CLI scripts

Converted all four scripts (`eval-runner.ts`, `generate-dataset.ts`, `generate-history.ts`, `generate-pdf.ts`) to typed TypeScript, including typing the PDFKit drawing helper functions (`drawBox`, `drawDiamond`, `drawArrow`) against the `PDFKit.PDFDocument` type. Verified the CLI runner end-to-end afterward with a live simulated run.

Step 5 — The React app, plus four bonus fixes discovered along the way

Converted every component to `.tsx` with real prop-type interfaces: `Dashboard`, `Comparator`, `Simulator`, `Dataset`, `App`, and the `useEvalRuns` hook. Four concrete improvements were bundled into this same pass, since it required touching every file anyway:

- **Error boundary** — added `ErrorBoundary.tsx`, a class component using `getDerivedStateFromError/componentDidCatch`, wrapping the whole app in `main.tsx` so a crash in any one tab shows a graceful fallback panel instead of a blank white screen.
- **Code-splitting** — converted the four tab components in `App.tsx` to `React.lazy()` + `Suspense`. The production bundle went from one 822KB JavaScript chunk to five separate chunks (the Recharts-heavy `Dashboard` split out to its own 395KB chunk, the other three tabs each under 8KB), so a user only downloads the code for the tab they're actually viewing.
- **Two real, pre-existing CSS bugs found and fixed** — the CSS custom property `--radius-full` was referenced by both the user avatar and status pill styles but was *never defined* anywhere in `index.css`, meaning those elements were rendering as squares/rectangles instead of pills for the entire life of the project until now. Separately, the class `.simulator-layout` was applied in both `Simulator.jsx` and `Comparator.jsx` but had no matching CSS rule at all, so the intended two-column workstation layout had never actually been styled as a grid. Both are now fixed and were visually confirmed correct in a live browser check.
- **Dead console-log coloring fixed** — the Simulator's terminal console tried to color-code log lines using Tailwind-style class names (`text-green-400`, etc.), but this project does not use Tailwind at all, so those classes had zero effect for the project's entire history. Replaced with real CSS classes (`.log-line.success/.error/.info`) tied to the app's actual color-variable system — confirmed working live (green/red/slate lines actually render correctly now).

Also added `aria-label` attributes and semantic roles to the search inputs, filter selects, the inspector drawer (`role="dialog" aria-modal`), and its close button, for basic screen-reader usability.

Step 6 — Expanded test suite

Converted the two existing test files to `.ts` and added two entirely new test files:

- `tests/database.test.ts` — spins up an isolated, temporary SQLite file per test run (via a new `RUNS_DB_PATH` environment variable added to `database.ts` specifically to make this possible, defaulting to the normal `runs.db` when unset) and verifies default-history seeding, that `insertRun` persists correctly including nested `ragConfig/metrics/testResults`, that results come back ordered by timestamp, and that the `commitId` `UNIQUE` constraint is actually enforced.
- `tests/server.test.ts` — uses Supertest to drive every route directly. To make this possible, `server.ts` was changed to export its app object and only call `app.listen()` when the file is actually executed as the program's entry point (detected by comparing `import.meta.url` against `process.argv[1]`), so a test file can import the same app without binding a real network port. Covers `config/runs` GETs, all four auth-failure and auth-success cases on the write endpoint, every validation-failure case (including the specific `ragConfig` gap that used to 500), and the eval-proxy's missing-prompt and missing-key error paths.

Final count: **35 tests across 4 files**, all passing — up from 8 tests in 1 file at the start of the session.

Step 7 — GitHub Actions CI workflow

Added `.github/workflows/eval-gate.yml`, triggered on every push and pull request to main. It checks out the repo, sets up Node, installs dependencies, then runs `typecheck` → `lint` → `test` → `build` → the real eval quality gate, capturing its output. On a pull request, it posts that output as a PR comment (■ passed or ■ failed, with the tail of the log), via `actions/github-script`, and explicitly fails the job — blocking the merge — if the eval gate itself failed. This is what makes the project's headline “CI/CD gatekeeper” claim literally true instead of aspirational; before this, there was no automation at all.

Step 8 — Husky git hooks

Installed Husky and added two hook scripts mirroring the CI pipeline locally: `.husky/pre-commit` runs `typecheck + lint`; `.husky/pre-push` runs `test + a real 10-item eval gate run`. Both were smoke-tested by executing them directly.

Step 9 — Docker

Wrote a multi-stage Dockerfile: the build stage installs all dependencies and runs `npm run build` to produce the static frontend; the runtime stage installs only production dependencies and runs `node server.ts`, which serves both the API and the built static assets on port 5189. `docker-compose.yml` wires this up with a named volume mounted at `/app/data` (with `RUNS_DB_PATH` pointed there) so the SQLite file survives container restarts, plus a `.dockerignore` to keep the build context small. Note: Docker itself was not available in this working environment, so the Dockerfile was reviewed carefully for correctness but the actual `docker build` has not been executed — this is flagged explicitly as unverified rather than claimed as tested.

Step 10 — OpenAPI specification

Wrote `openapi.yaml` (OpenAPI 3.0.3) documenting all four routes: request/response schemas for `EvalRun`, `RagConfig`, `RunMetrics`, and `TestResult`; every status code each route can actually return; and the bearer-token security scheme, including the deliberate distinction between `/api/eval`'s optional auth and `/api/runs/add`'s fail-closed auth. Validated with `@redocly/cli lint`, which confirmed it as a fully valid spec; the only remaining warnings are stylistic (missing explicit 4xx documentation on two GET routes that, checked

against the actual implementation, genuinely have no 4xx code path) and were deliberately left rather than papered over with a fake response.

Step 11 — Documentation updates & final verification

Updated `README.md` and `PROJECT_EXPLANATION.md` to fix stale file-extension references (`.js/.jsx` → `.ts/.tsx`) and added new sections documenting the development-check commands, the Docker workflow, and the OpenAPI reference. Then ran a complete final pass: `typecheck`, `lint`, `test` (35/35 passing), and `build` all green, plus a full live browser walkthrough — clicked through all four tabs, ran an actual end-to-end simulated evaluation from the Simulator tab, and confirmed both CSS fixes rendering correctly (round avatar, properly-laid-out two-column Simulator view, correctly colored terminal log lines).

Rating after this phase: **9/10**, up from 7.5 — driven mainly by a new “engineering rigor” dimension (real CI, real hooks, Docker, OpenAPI) that didn't exist as a category before, plus meaningful gains in code quality (full typing, no more duplication) and testing (6/10 → 8.5/10).

7.4 Vercel Deployment Diagnosis & Fix

During a live screen-share, the deployed demo (solar-six-tan.vercel.app) hit an error in the Simulator console when trying a real Gemini call: `CONNECTION FAILURE EXPOSED LOUDLY: Unexpected token 'T', "The page c"... is not valid JSON`. That specific message — `response.json()` choking on text starting with “The page c...” — is the signature of a `fetch()` call getting back an HTML error page instead of a JSON API response.

Diagnosis

Rather than guess, the live deployment was queried directly to isolate whether the problem was in the app's code or in how (or whether) it was actually deployed:

```
GET https://solar-six-tan.vercel.app/api/config → HTTP 404, header x-vercel-error: NOT_FOUND, body "The page could not be found".
```

That response header is Vercel's own platform-level routing error — not anything coming from the Express app at all. The request never reached `server.ts`'s route handlers. Root cause: the repository has no `vercel.json` and no top-level `api/` directory, so Vercel's zero-config detection for a Vite project only ever knew to serve the static `dist/` build; it had no configuration telling it that `server.ts` should be invoked as a serverless function at all. This gap pre-dated this session's TypeScript work entirely — it was confirmed directly against the live URL, not against local code.

This also explains why the Dashboard still *appeared* to work on the live site: `useEvalRuns.ts`'s `fetchHistory()` catches the failed `/api/runs` call and silently falls back to the static `runs_history.json` bundled into the build — so charts and history looked populated, but were never actually reading from the live backend. Only the Simulator's real-API path surfaced the failure loudly, exactly as it was designed to do.

A second, deeper issue flagged at the same time

Even after wiring up routing, SQLite fundamentally does not suit Vercel's serverless model: the Lambda filesystem is read-only everywhere except `/tmp`, and `/tmp` itself is neither shared across concurrent function instances nor durable across cold starts. This was surfaced to the user explicitly rather than silently worked around, since it affects whether “fixing the routing” actually means “fully fixed” or just “works per-request, doesn't crash, but writes don't durably persist.”

The fix implemented (scoped, on explicit approval, to routing only)

- Created `api/index.ts` as a Vercel serverless endpoint. It sets `process.env.RUNS_DB_PATH = '/tmp/runs.db'` if not already set (so SQLite has somewhere writable to open a file at all under Vercel's read-only filesystem), then dynamically imports `database.ts` and `server.ts`, calls `initDb()` once at module load (so tables exist and default history is seeded before any request is handled), and exports the Express app object as the default export — which is exactly the shape Vercel's `@vercel/node` builder expects for an Express app used as a serverless function.
- Added `vercel.json` with a rewrite rule sending every `/api/(.*)` request to the `/api` function (which Vercel auto-detects from `api/index.ts`), plus an explicit `includeFiles` directive for `src/data/**` so the bundler is guaranteed to package the golden dataset and run-history JSON files the server reads at runtime via a computed `fs.readFileSync` path (a pattern static bundling tools can sometimes miss without an explicit hint).

Verified without deploying anything: imported `api/index.ts`'s default export directly in a local script and confirmed it is genuinely an Express app (has `.use` and `.listen`) that initializes the database without ever

binding a port, then drove it with Supertest exactly the way Vercel would invoke it — GET /api/config returned 200 {hasServerKey: false}, and GET /api/runs returned 200 with the 5 seeded historical runs. This confirms the fix is correct *as code*; it has not yet been pushed or deployed to Vercel, since that is a decision left to the project owner.

8. Known Limitations & Honest Caveats

In the same spirit as the project's own Transparency Log, here is a complete, unvarnished list of what is still imperfect or unverified as of this session:

- **SQLite is not a durable data store on Vercel serverless.** Even with the routing fix in place, writes to `/tmp/runs.db` only last for the lifetime of one warm Lambda instance. A real production deployment needing persistent run history should use a hosted database (e.g. Postgres, Turso, or Neon) instead.
- **The Docker build has been reviewed but not executed.** Docker was not available in this working environment, so `docker compose up --build` should be run once locally to confirm the multi-stage build actually succeeds end to end before relying on it.
- **The Vercel routing fix has not been deployed.** The code exists and was verified locally, but nothing has been pushed or redeployed — the live demo will keep 404ing on `/api/*` until that happens.
- **The original `PROJECT_EXPLANATION.pdf` (PDFKit-generated) has not been regenerated** since the TypeScript migration, so a couple of its file-path references still describe the pre-migration `.js` layout. This new document supersedes it for anyone who needs the current, accurate picture; regenerating the original is a one-command follow-up (`node scripts/generate-pdf.ts`).
- **The Bring-Your-Own-Key Gemini key is stored in `localStorage` in plaintext** on the client. Acceptable for a personal demo where the user pastes their own free-tier key, but not a pattern to carry into anything handling real user credentials.
- **The eval SLA gate is a genuine, randomized check — it can legitimately fail on small subsets.** This was observed directly during verification (a 3-item simulated subset failed the faithfulness gate at 89.2%, just under the 90% bar). That is the gate working exactly as designed against randomized simulation data, not a bug — but it does mean a CI run on a small sample size can occasionally be flaky by nature.

9. How to Run Everything

Command	What it does
<code>npm install</code>	Install all dependencies, including Husky's git hooks via the postinstall "prepare" script.
<code>npm run dev</code>	Start the Vite dev server on port 5188 (proxies /api to the backend on 5189).
<code>npm start</code>	Run the Express + SQLite backend directly (server.ts) on port 5189.
<code>npm run eval</code>	Run the CLI quality-gate check against the default (simulated) config, 100-item subset.
<code>npm run eval -- --model=gemini-1.5-flash --subset=10 --apiKey=YOUR_KEY</code>	Run the CLI check with real Gemini API calls against 10 items.
<code>npm run typecheck</code>	Type-check both the frontend and backend TypeScript projects (no emit).
<code>npm run lint</code>	Run ESLint across the whole codebase.
<code>npm run test</code>	Run the full Vitest suite (35 tests: scoring logic, SQLite, and API routes).
<code>npm run build</code>	Typecheck, then produce the optimized, code-split production frontend bundle.
<code>docker compose up --build</code>	Build and run the whole app (frontend + backend) in one container on port 5189, with a persistent named volume for the SQLite file.

Git hooks (installed automatically): `pre-commit` runs `typecheck` + `lint`; `pre-push` runs the full test suite plus a real 10-item eval gate — so most of the CI pipeline is already enforced before code even reaches GitHub.

10. Final Assessment

Dimension	Score	Why
Concept & scope	9 / 10	A genuinely useful, well-scoped idea, made stronger by unusually honest documentation of what's real vs. simulated.
Code quality	9 / 10	Full strict-mode TypeScript everywhere, zero logic duplication between the CLI and web paths, code-split bundle, two real pre-existing CSS bugs fixed.
Security	8 / 10	Write endpoint fails closed without a configured secret; input validation closes the crash-to-500 gap. BYOK key storage in localStorage remains demo-grade, not production-grade.
Testing	8.5 / 10	35 tests spanning scoring logic, SQLite behavior, and every API route/auth/validation path. No direct React component tests yet.
Engineering rigor	9 / 10	Real CI (typecheck/lint/test/build/eval-gate + PR comments), matching git hooks, Docker, and a validated OpenAPI spec — the project's "CI/CD gatekeeper" claim is now literally true.
Documentation	8.5 / 10	README and architecture docs kept in sync with the codebase; this document adds a complete, dated engineering record on top.

Overall: 9/10, up from an initial 7.5/10 at the start of this session. What's left to close the gap to 9.5–10: actually deploy the Vercel routing fix (or swap the run-history store to a durable hosted database for real production use), execute a real docker build to confirm the container image, and regenerate the original PDFKit-based PROJECT_EXPLANATION.pdf so both documents agree.